Codebase-Wide Version Migration and Refactoring Agent: Literature Survey and Requirement Analysis

Verril Vaz (SRN: R23EN179)
School of CSE, REVA University
Faculty Guide: Dr. Nimrita Koul
Center for Artificial Intelligence & Machine Learning
September 2026

***Abstract*: Upgrading a large codebase to a new language or library version is a recurring maintenance cost. Breaking changes make the task hard to automate because a single deprecated API may be used at hundreds of call sites spread over many modules. This report presents the literature survey and requirement analysis for a project that will build an autonomous migration agent. The survey covers API evolution, rule-based refactoring tools, large language models for code, repository-level code generation, tool-using agents, and recent LLM-based migration systems. It finds that existing work either depends on hand-written transformation rules that do not generalise, or gives a language model a flat context that does not scale to multi-file repositories, and that few systems combine static call-graph analysis, persistent graph memory and test-driven self-correction in a single loop. From these gaps the report derives functional and non-functional requirements, evaluation metrics, environment and benchmark needs, and a risk register for the agent to be developed.**

***Keywords*: code migration, API evolution, abstract syntax tree, call graph, LLM agents, program repair, static analysis, refactoring.**

I. INTRODUCTION

A. Background and Motivation

Software systems routinely outlive the language versions and library releases they were written for. Python 2 reached end of life in January 2020 [1]. The lib2to3 package, which shipped with Python 3 to assist the transition, was deprecated in Python 3.11 and removed in Python 3.13 [2]. SQLAlchemy 2.0 made select() constructs the primary way of issuing database statements and moved the older Query interface to legacy status [3]. Each of these is a breaking change: code that ran correctly before the upgrade either fails or behaves differently after it.

In a small project such an upgrade takes a few edits. In a multi-module system it does not. A deprecated function may be called from hundreds of sites through import aliases, wrapper functions and inherited methods, and a change to one signature alters the contract that every caller relies on. Three properties make the task hard to automate. First, finding every affected call site requires resolving names across files, which plain text search cannot do reliably. Second, edits interact, so the order in which modules are changed decides whether an intermediate state is consistent. Third, correctness can only be confirmed by running the test suite, and the failures it reports usually appear far from the edit that caused them.

Large language models (LLMs) can rewrite a code fragment that is shown to them, but a whole repository does not fit in a prompt. An agent that works from a flat, growing conversation history loses track of what has been changed, what is pending and what has been verified. The project reported here therefore couples an LLM with static analysis, an explicit graph memory, and a test-driven correction loop.

B. Problem Statement

Given a migration specification and a multi-module Python repository, build an agent that (i) statically identifies every call site affected by the specification before any edit is made, (ii) plans and applies the changes in an order that avoids circular regressions, (iii) tracks the state of every file over a long execution without exceeding a token budget, and (iv) runs linters and tests after each batch of edits and repairs the reported failures until the test suite passes.

C. Objectives and Expected Outcomes

The project has four technical objectives. **O1**: automated dependency and call-graph mapping using ASTs, so that all breaking call sites are known before editing. **O2**: a long-horizon planning and execution engine that orders, batches and applies changes across modules. **O3**: graph-based memory that records file modification states, pending dependency fixes and context summaries within a fixed token budget. **O4**: interleaved verification and self-correction using linter and test runs, stack-trace parsing and targeted patches.

The expected outcomes are an agent framework, a unified migration patch, structured JSON trajectory and state logs, an empirical benchmarking report over multi-file migration tasks, and a research paper.

D. Scope and Organisation of this Report

This report covers the first project milestone. Section II surveys related work and identifies research gaps. Section III analyses requirements: stakeholders, functional and non-functional requirements, software and hardware needs, benchmark and data needs, evaluation metrics, risks, the proposed architecture and the project plan. Section IV concludes.

II. LITERATURE SURVEY

The survey draws on the ACM Digital Library, IEEE Xplore, arXiv and Google Scholar. Search terms combined API evolution, breaking changes, automated

refactoring, code migration, repository-level code generation, LLM agents and software engineering benchmarks. Peer-reviewed venues were preferred, and recent preprints were included where they define benchmarks or systems directly relevant to migration. The works are grouped by theme below.

A. API Evolution and Breaking Changes

Dig and Johnson studied the histories of several libraries and found that the large majority of changes that break client code are refactorings such as renames and signature changes, and not behavioural redesigns [4]. This finding supports a design in which most migrations are treated as structured, mechanical transformations. Henkel and Diwan's CatchUp records refactorings as they are performed in an editor and replays them on client code [5], an early example of migration by captured transformation.

Lamothe et al. reviewed the API evolution literature and reported that open challenges include identifying the factors that drive API changes, the absence of uniform benchmarks for evaluating research, and the limited understanding of API evolution across programming languages [6]. For Python specifically, PCART detects and repairs API parameter compatibility issues that arise when a third-party library is upgraded [7]. PCART concentrates on parameter-level incompatibilities and does not plan a multi-module upgrade.

B. Rule-Based Refactoring and Migration Tools

Fowler catalogues behaviour-preserving refactorings and the discipline of applying them in small steps under test [8]. Tooling for automated, large-scale change followed. Coccinelle expresses collateral evolutions of Linux device drivers as semantic patches [9], and ClangMR applies refactorings over Clang ASTs in a MapReduce framework to change very large C++ codebases [10]. For Python, 2to3 applies a set of fixers to convert Python 2 source [2], pyupgrade and the corresponding Ruff rules rewrite syntax to newer language versions [11], [12], and LibCST provides a concrete syntax tree that preserves comments and formatting, on which custom codemods can be written [13].

These tools are precise and reproducible, which makes them the right foundation for the mechanical part of a migration. Their limits are that each transformation must be authored by hand for the migration at hand, that they generally decide from local syntax and cannot exercise judgement on ambiguous sites, and that they do not check whether the migrated program still passes its tests.

C. Language Models for Code and Code Translation

Codex established that LLMs fine-tuned on code can synthesise functionally correct programs, measured by unit tests [14]. Studies of translation show the risk of using such models without checks: Pan et al. analysed bugs introduced by LLMs when translating code between languages and classified their root causes [15]. Version awareness is a related weakness. VersiCode covers 300 libraries across more than 2,000 versions and evaluates version-specific completion and editing [16], and GitChameleon provides 116 executable Python problems, each tied to a library version; GPT-4o reaches only 39.9 percent pass@10, or 43.7 percent when given error feedback [17]. Together these results suggest that a model cannot be trusted to know the right API for a given version, and that the migration specification and execution feedback must supply that knowledge.

D. Repository-Level Code Generation and Context Selection

RepoCoder retrieves snippets from the repository and iterates retrieval and generation to complete code that depends on other files [18]. RepoBench evaluates retrieval, completion and pipeline settings that require cross-file context [19]. Both address the selection of relevant context under a bounded prompt, which is also central here. They use similarity-based retrieval and target completion of a single location. A migration must instead edit many related locations and keep them mutually consistent, where dependency structure is a more direct signal of relevance than textual similarity.

E. LLM Agents: Reasoning, Tools, Feedback and Memory

ReAct interleaves reasoning traces with tool actions so that the model can gather information and revise its plan [20]. Toolformer trains models to decide when to call external tools [21]. Reflexion stores verbal reflections on past failures in an episodic memory and uses them in later attempts [22], and Self-Refine iterates between generation and self-generated feedback [23]. MemGPT manages a tiered memory so that a model with a fixed context window can work over longer histories [24]. These works give the building blocks of the proposed loop: tool use, feedback-driven revision and memory outside the prompt. None of them defines a memory schema specific to code, such as per-file migration state over a dependency graph.

F. Agents for Software Engineering Tasks

SWE-bench contains 2,294 task instances drawn from real issues and pull requests in 12 popular Python repositories, with tests as the success criterion [25]. SWE-agent shows that the design of the agent-computer interface, meaning the commands and feedback an agent is given, strongly affects performance, and reports a 12.5 percent pass@1 rate on SWE-bench [26]. AutoCodeRover combines LLM analysis of an issue with program-structure-aware code search over classes and methods [27]. Agentless questions the need for complex agents and uses a fixed pipeline of localisation, repair and patch validation, reaching 32.00 percent on SWE-bench Lite at about 0.70 US dollars per issue [28].

The Agentless result is a useful caution: a structured, mostly deterministic pipeline can outperform an open-ended agent. This project adopts the same principle by giving static analysis and codemods the deterministic work and reserving the LLM for judgement and repair. These systems, however, resolve one issue that is usually localised to a few files. A codebase-wide migration applies one change pattern to many places and requires a plan over the whole repository.

G. LLM-Based Code Migration

CodePlan is the closest prior work. It treats repository-level coding as a planning problem and

synthesises a chain of edits using incremental dependency analysis, change may-impact analysis and adaptive planning around LLM calls. It was evaluated on package migration in C# and on temporal code edits in Python, on repositories requiring changes to between 2 and 97 files [29]. At Google, a system identifies locations to change, generates edits with LLMs fine-tuned on internal code, and validates them through compilation and test execution [30]. MigrationBench defines repository-level migration from Java 8 to Java 17 and 21 over 5,102 repositories, with a 300-repository subset, and proposes an agentic baseline [31].

These works confirm that combining static analysis, planning and execution feedback is the right direction. They leave room for the present project in several respects. The Google system is proprietary and depends on models tuned on internal code. MigrationBench targets Java. CodePlan was not evaluated on Python library upgrades checked against a project test suite. None of the three reports a token and step budget as a first-class evaluation axis alongside completeness and test pass rate.

TABLE I
COMPARATIVE SUMMARY OF RELATED WORK

Work	Technique	Limitation for this project
2to3, LibCST codemods, Ruff	Hand-written syntax rules	Rules per migration; no test feedback
CatchUp, Coccinelle, ClangMR	Replayed refactorings; semantic patches; AST tools	Pattern must be specified; little judgement on ambiguous sites
RepoCoder, RepoBench	Retrieval of cross-file context	Completion of one location, not ordered multi-file edits
SWE-agent, AutoCodeRover	Agent-computer interface; AST-aware code search	Single-issue fixes; flat trajectory; no persistent state graph
Agentless	Localise, repair, validate	No long-horizon plan over many files
CodePlan	Dependency analysis and adaptive planning	C# migration and Python temporal edits only
Google migration system	LLM edits with compile and test validation	Proprietary; internally tuned models
MigrationBench	Java 8 to 17 benchmark	Java only
VersiCode, GitChameleon	Version-specific code benchmarks	Snippet-level, not repository migration

H. Research Gaps

The survey leads to five gaps that the project addresses.

G1. Impact mapping before editing. Static call-graph analysis is rarely used to compute the full set of breaking call sites, and their transitive callers, before an LLM edits any file in a Python codebase.

G2. Structured long-horizon memory. Agents keep history as prompt text. A per-file state graph (pending, analysed, edited, verified, failed) with summaries under a token budget is not standard practice.

G3. Failure attribution. Test failures surface far from the offending edit. Mapping a stack trace back through the call graph to the edit or contract that caused it, and patching that location, is underexplored.

G4. Ordering without circular regressions. Little work describes ordering and batching edits over cyclic dependencies so that repairing one module does not undo another.

G5. Evaluation. Lamothe et al. report the lack of uniform benchmarks [6]. Python migration benchmarks that measure completeness, test pass rate and token and step overhead together are missing.

III. REQUIREMENT ANALYSIS

A. Stakeholders and Use Cases

The primary users are developers or maintainers who need to upgrade a repository, researchers who evaluate the agent, and the faculty reviewers of the project. Four use cases follow. **UC1**: run a migration, given a specification and a repository, and receive a patch. **UC2**: run in dry-run mode to inspect the call-site map and the plan without editing. **UC3**: review the patch together with the trajectory log to understand every decision. **UC4**: run the benchmark and ablation suite and obtain aggregate metrics.

B. Functional Requirements

Priorities follow the MoSCoW convention: M (must), S (should), C (could). Table II lists the requirements, grouped by the four objectives.

TABLE II
FUNCTIONAL REQUIREMENTS

ID	Requirement	Pri.
FR1	Accept a migration specification (a rule file or natural-language description, with optional codemod rules) and a repository path.	M
FR2	Parse every Python source file into an AST or concrete syntax tree, and record files that cannot be parsed instead of aborting.	M
FR3	Build an import graph and symbol table that resolve aliases, star imports and re-exports.	M
FR4	Build a call graph and list every call site of a deprecated API, including sites reached through aliases and wrappers, with file, line and enclosing function.	M
FR5	Compute the impact set: transitive callers whose contract may change, and sites that cannot be resolved statically.	M
FR6	Produce an edit plan by ordering files topologically over the dependency graph, collapsing cycles into strongly connected components, and grouping files into batches.	M
FR7	Maintain a persistent graph memory with a state for every file (pending, analysed, edited, verified, failed) and record pending dependency fixes.	M
FR8	Build each LLM prompt from the current node, its neighbours, open failures and stored summaries within a configurable token budget.	M
FR9	Apply edits through deterministic codemods first and LLM-proposed patches for residual cases; reject any patch that does not apply or leaves a file syntactically invalid.	M
FR10	Run the linter and the affected tests after each batch, and the full test suite at the end, inside an isolated container.	M
FR11	Parse stack traces and linter output, map failing frames to graph nodes, attribute the failure to a breaking contract, and generate a targeted fix.	M

ID	Requirement	Pri.
FR12	Checkpoint with Git after each batch, roll back a failed batch, bound the number of retries, and detect repeated or oscillating patches.	M
FR13	Emit a unified .patch file and structured JSON logs of the call-graph, edit order, failures and corrective actions.	M
FR14	Provide an evaluation harness that computes per-task and aggregate metrics and supports ablation switches.	S
FR15	Provide a command-line interface and configuration file, including a dry-run mode.	S
FR16	Support pluggable migration specifications so that a new migration needs no change to the core engine.	S

C. Non-Functional Requirements

TABLE III
NON-FUNCTIONAL REQUIREMENTS

ID	Requirement	Target
NFR1	Reproducibility: pinned dependency versions, fixed seeds, temperature 0, and all prompts and tool outputs logged.	Same task gives the same patch within logged variance
NFR2	Isolation: all test and tool execution in a container with resource limits; the original repository is never edited in place.	No host modification
NFR3	Efficiency: configurable ceilings on tokens, steps and wall-clock time per task.	Hard stop at ceiling
NFR4	Scalability: analysis time grows roughly linearly with repository size.	1,000 files analysed in minutes (to be refined after profiling)
NFR5	Correctness of edits: formatting and comments outside changed regions are preserved.	Diff limited to changed sites
NFR6	Extensibility: analysis, planning, memory and verification are separate modules with defined interfaces.	Replace a module without changes elsewhere
NFR7	Observability: logs follow a versioned JSON schema.	Every decision traceable
NFR8	Portability: develops on Windows, runs in Linux containers.	Same results on both

D. Software and Hardware Requirements

Table IV lists the intended toolchain. Python 3.12 is the implementation language. The standard ast module [32] is sufficient for parsing valid Python 3 source, LibCST [13] is needed where edits must preserve formatting, and tree-sitter [33] offers error-tolerant parsing. NetworkX [34] holds the dependency and call graphs, and strongly connected components are found with Tarjan's algorithm [35]. Tests run under pytest [36] and lint checks under Ruff [12]. A symbol resolver such as Jedi or Pyright will be evaluated for accuracy on aliased and inherited calls.

TABLE IV
SOFTWARE AND HARDWARE REQUIREMENTS

Item	Requirement
Language	Python 3.12
Parsing	ast, LibCST, tree-sitter
Graph and analysis	NetworkX; Jedi or Pyright (to be evaluated)
Test and lint	pytest, Ruff, flake8
Version control	Git
Sandbox	Docker with CPU, memory and network limits
LLM access	Hosted API or local open model (to be decided)
Logging	JSON, versioned schema
Hardware	Multi-core CPU, 16 GB RAM recommended (8 GB minimum), about 50 GB disk for repositories and images; GPU only if a local model is used

E. Data and Benchmark Requirements

Three migration families are proposed: (M1) SQLAlchemy 1.4 to 2.0, which replaces legacy Query usage with select() constructs [3]; (M2) Python 2 to Python 3; and (M3) Python 3.8 to 3.12, which must handle modules and aliases removed in 3.12, for example distutils, imp, asyncore and the deprecated unittest assertion aliases [37]. M1 is the primary case because it exercises cross-file call sites most strongly. M2 needs special care because the Python 3 ast module cannot parse Python 2 source, so a fallback parser is required for that family.

Every benchmark task must satisfy four conditions: the repository has a test suite that passes before migration under the old version; an oracle detector, independent of the agent's own call-site finder, counts deprecated usages before and after; the repository spans several modules with cross-file dependencies; and a reference migration exists where possible. The problem statement calls for evaluation over 510 multi-file tasks. No public Python benchmark of that size exists, since related benchmarks are either Java-based [31] or snippet-level [17]. Tasks will therefore be assembled from real repositories with historical upgrade commits and from synthetic tasks created by controlled injection of deprecated patterns. The final task count will be confirmed with the faculty guide.

F. Evaluation Metrics

Let B be the number of deprecated usages found by the oracle before migration and A the number remaining afterwards. Let T be the number of tests in the post-migration suite and P the number that pass.

$$\text{Migration Completeness (MC)} = (B - A) / B \times 100\%$$

$$\text{Post-Migration Test Pass Rate (TPR)} = P / T \times 100\%$$

Token overhead is the total input and output tokens per task, and step overhead is the number of agent iterations and tool calls per task. Secondary metrics are task success rate (MC and TPR both 100 percent), patch precision (share of edited sites that were truly deprecated), mean repair iterations to a green suite, and wall-clock time. Regressions are counted separately as tests that passed before migration and fail after it.

Baselines will be a single-shot LLM, a flat-context ReAct-style agent, and where reproducible a published repository-level system. Ablations remove one component at a time: static impact analysis, graph memory, interleaved verification, and codemods (LLM-only against hybrid).

G. Constraints, Assumptions and Risks

TABLE V
RISK REGISTER

Risk	Mitigation
Python 2 source cannot be parsed by the Python 3 ast module	Fallback parser (lib2to3 grammar in Python 3.12, or tree-sitter with error recovery)
Dynamic features (getattr, eval, monkeypatching) defeat static call resolution	Report unresolved sites; rely on test execution to expose them
Weak test suites give false confidence	Select repositories by test coverage; report coverage
LLM cost and nondeterminism	Token ceilings, response caching, temperature 0, logged prompts
Benchmark size of 510 tasks	Synthetic task generation; confirm target with guide
Flaky tests	Baseline runs before migration; rerun failing tests
Oscillating repairs	Retry bound, patch-history check, batch rollback
Training-data contamination	Include recent and synthetic tasks; report by task source

H. Proposed Architecture

Fig. 1 shows the proposed pipeline. The *analyser* parses the repository and builds the import graph, call graph and impact set. The *planner* orders files, collapses cycles into components and forms batches. The *executor* applies deterministic codemods and, for residual cases, LLM-proposed patches built from a token-bounded context. The *verifier* runs the linter and tests in a container and returns parsed failures to the executor. The *graph memory* is shared by all stages and holds file states, pending fixes and summaries. A logger writes every decision as JSON, and the final stage emits the patch.

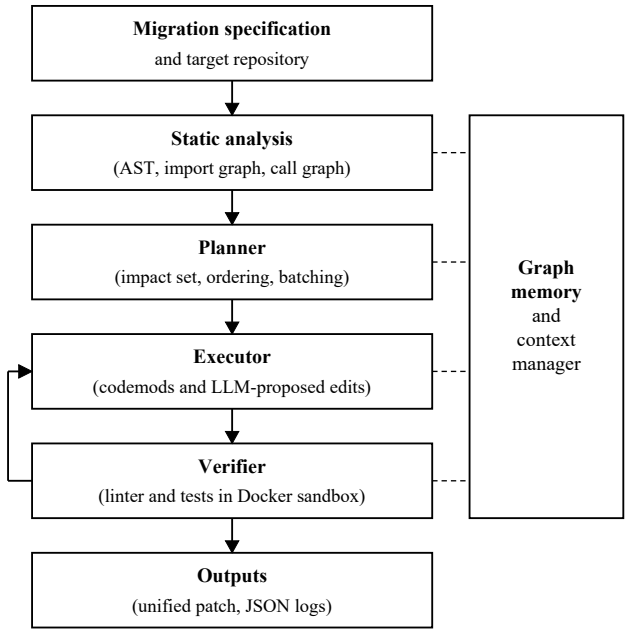


Fig. 1. Proposed architecture. The left arrow returns verifier failures to the executor; the dashed lines show access to the shared graph memory.

I. Project Plan

TABLE VI
PROJECT PHASES

Phase	Activity	Output
1	Literature survey and requirement analysis	This report
2	Analysis layer: call-site finder, import and call graphs, oracle detector	Analyser module
3	Planner and graph memory with context budgeting	Planner and memory modules
4	Sandbox, test and lint runner, failure attribution and repair loop	Working agent
5	Benchmark construction, baselines, ablations	Metrics and logs
6	Patch, benchmarking report and paper	Final deliverables

IV. CONCLUSION

The survey shows that rule-based tools provide precision but not generality, that LLM agents provide flexibility but degrade over long, multi-file tasks, and that recent migration systems combine both without fully addressing persistent state, failure attribution and cost reporting for Python. The requirement analysis converts these gaps into sixteen functional and eight non-functional requirements, a metric set of completeness, test pass rate and token and step overhead, and a risk register. The next phase is implementation of the analysis layer, beginning with the SQLAlchemy 2.0 migration as the primary case.

REFERENCES

[1] Python Software Foundation, "Sunsetting Python 2." [Online]. Available: https://www.python.org/doc/sunset-python-2/

[2] Python Software Foundation, "2to3: Automated Python 2 to 3 code translation," Python 3.12 Documentation. [Online]. Available: https://docs.python.org/3.12/library/2to3.html

[3] SQLAlchemy Project, "Migrating to SQLAlchemy 2.0," SQLAlchemy 2.0 Documentation. [Online]. Available: https://docs.sqlalchemy.org/en/20/changelog/migration_20.html

[4] D. Dig and R. Johnson, "How do APIs evolve? A story of refactoring," *J. Softw. Maint. Evol.: Res. Pract.*, vol. 18, no. 2, pp. 83-107, 2006.

[5] J. Henkel and A. Diwan, "CatchUp! Capturing and replaying refactorings to support API evolution," in *Proc. 27th Int. Conf. Softw. Eng. (ICSE)*, 2005.

[6] M. Lamothe, Y.-G. Guéhéneuc, and W. Shang, "A systematic review of API evolution literature," *ACM Comput. Surv.*, vol. 54, no. 8, art. 171, 2021.

[7] S. Zhang, G. Xiao, J. Wang, H. Lei, G. He, Y. Liu, and Z. Zheng, "PCART: Automated repair of Python API parameter compatibility issues," *IEEE Trans. Softw. Eng.*, vol. 52, no. 3, pp. 723-753, 2026.

[8] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.

[9] Y. Padioleau, J. L. Lawall, R. R. Hansen, and G. Muller, "Documenting and automating collateral evolutions in Linux device drivers," in *Proc. EuroSys*, 2008.

[10] H. K. Wright, D. Jasper, M. Klimek, C. Carruth, and Z. Wan, "Large-scale automated refactoring using ClangMR," in *Proc. IEEE Int. Conf. Softw. Maint. (ICSM)*, 2013.

[11] A. Sottile, "pyupgrade: A tool to automatically upgrade syntax for newer versions of Python," GitHub repository. [Online]. Available: https://github.com/asottile/pyupgrade

[12] Astral Software Inc., "Ruff: An extremely fast Python linter and code formatter," GitHub repository. [Online]. Available: https://github.com/astral-sh/ruff

[13] Instagram, "LibCST: A concrete syntax tree parser and serializer library for Python," GitHub repository. [Online]. Available:

https://github.com/Instagram/LibCST

[14] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[15] R. Pan *et al.*, "Lost in translation: A study of bugs introduced by large language models while translating code," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, 2024.

[16] T. Wu *et al.*, "VersiCode: Towards version-controllable code generation," arXiv:2406.07411, 2024.

[17] N. Islah *et al.*, "GitChameleon: Unmasking the version-switching capabilities of code generation models," arXiv:2411.05830, 2024.

[18] F. Zhang *et al.*, "RepoCoder: Repository-level code completion through iterative retrieval and generation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2023.

[19] T. Liu, C. Xu, and J. McAuley, "RepoBench: Benchmarking repository-level code auto-completion systems," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[20] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[21] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[22] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[23] A. Madaan *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[24] C. Packer *et al.*, "MemGPT: Towards LLMs as operating systems," arXiv:2310.08560, 2023.

[25] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[26] J. Yang *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[27] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*, 2024.

[28] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," *Proc. ACM Softw. Eng. (FSE)*, 2025.

[29] R. Bairi *et al.*, "CodePlan: Repository-level coding using LLMs and planning," *Proc. ACM Softw. Eng. (FSE)*, vol. 1, art. 31, 2024.

[30] C. Ziftci, S. Nikolov, A. Sjövall, B. Kim, D. Codecasa, and M. Kim, "Migrating code at scale with LLMs at Google," in *Companion Proc. 33rd ACM Int. Conf. Found. Softw. Eng. (FSE Companion)*, 2025, pp. 162-173.

[31] L. Liu *et al.*, "MigrationBench: Repository-level code migration benchmark from Java 8," arXiv:2505.09569, 2025.

[32] Python Software Foundation, "ast: Abstract syntax trees," Python 3.12 Documentation. [Online]. Available: https://docs.python.org/3.12/library/ast.html

[33] M. Brunsfeld *et al.*, "Tree-sitter: An incremental parsing system for programming tools," GitHub repository. [Online]. Available: https://github.com/tree-sitter/tree-sitter

[34] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python in Science Conf. (SciPy)*, 2008, pp. 11-15.

[35] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM J. Comput.*, vol. 1, no. 2, pp. 146-160, 1972.

[36] pytest-dev, "pytest documentation." [Online]. Available: https://docs.pytest.org

[37] Python Software Foundation, "What's new in Python 3.12," Python 3.12 Documentation. [Online]. Available: https://docs.python.org/3.12/whatsnew/3.12.html